

In some cases you'll want to use APIs that use an **async** pattern within an event handler. For example, you might use a [Button](#) in an [AppBar](#) to display a file picker and interact with it. However, many of the file picker APIs are asynchronous. They have to be called within an **async**/awaitable scope, and the compiler will enforce this. So what you can do is add the **async** keyword to your event handler such that the handler is now **async void**. Now your event handler is permitted to make **async**/awaitable calls.

For an example of user-interaction event handling using the **async** pattern, see [File access and pickers](#) (part of the [Create your first Windows Runtime app using C# or Visual Basic](#) series). See also [\[Call asynchronous APIs in C\]](#).

Adding event handlers in code

XAML is not the only way to assign an event handler to an object. To add event handlers to any given object in code, including to objects that are not usable in XAML, you can use the language-specific syntax for adding event handlers.

In C#, the syntax is to use the `+=` operator. You register the handler by referencing the event handler method name on the right side of the operator.

If you use code to add event handlers to objects that appear in the run-time UI, a common practice is to add such handlers in response to an object lifetime event or callback, such as [Loaded](#) or [OnApplyTemplate](#), so that the event handlers on the relevant object are ready for user-initiated events at run time. This example shows a XAML outline of the page structure and then provides the C# language syntax for adding an event handler to an object.

XAML

```
<Grid x:Name="LayoutRoot" Loaded="LayoutRoot_Loaded">
    <StackPanel>
        <TextBlock Name="textBlock1">Put the pointer over this text</TextBlock>
        ...
    </StackPanel>
</Grid>
```

C#

```
void LayoutRoot_Loaded(object sender, RoutedEventArgs e)
{
    textBlock1.PointerEntered += textBlock1_PointerEntered;
    textBlock1.PointerExited += textBlock1_PointerExited;
}
```

Note A more verbose syntax exists. In 2005, C# added a feature called delegate inference, which enables a compiler to infer the new delegate instance and enables the previous, simpler syntax. The verbose syntax is functionally identical to the previous example, but explicitly creates a new delegate instance before registering it, thus not taking advantage of delegate inference. This explicit syntax is less common, but you might still see it in some code examples.

C#

```
void LayoutRoot_Loaded(object sender, RoutedEventArgs e)
{
    textBlock1.PointerEntered += new
    PointerEventHandler(textBlock1.PointerEntered);
    textBlock1.PointerExited += new
    MouseEventHandler(textBlock1.PointerExited);
}
```

There are two possibilities for Visual Basic syntax. One is to parallel the C# syntax and attach handlers directly to instances. This requires the **AddHandler** keyword and also the **AddressOf** operator that dereferences the handler method name.

The other option for Visual Basic syntax is to use the **Handles** keyword on event handlers. This technique is appropriate for cases where handlers are expected to exist on objects at load time and persist throughout the object lifetime. Using **Handles** on an object that is defined in XAML requires that you provide a **Name / x:Name**. This name becomes the instance qualifier that is needed for the *Instance.Event* part of the **Handles** syntax. In this case you don't need an object lifetime-based event handler to initiate attaching the other event handlers; the **Handles** connections are created when you compile your XAML page.

VB

```
Private Sub textBlock1_PointerEntered(ByVal sender As Object, ByVal e As
    PointerRoutedEventArgs) Handles textBlock1.PointerEntered
    ' ...
End Sub
```

Note Visual Studio and its XAML design surface generally promote the instance-handling technique instead of the **Handles** keyword. This is because establishing the event handler wiring in XAML is part of typical designer-developer workflow, and the **Handles** keyword technique is incompatible with wiring the event handlers in XAML.

In C++/CX, you also use the += syntax, but there are differences from the basic C# form:

- No delegate inference exists, so you must use **ref new** for the delegate instance.
- The delegate constructor has two parameters, and requires the target object as the first parameter. Typically you specify **this**.
- The delegate constructor requires the method address as the second parameter, so the **&** reference operator precedes the method name.

C++/WinRT

```
textBlock1().PointerEntered({this, &MainPage::TextBlock1_PointerEntered });
```

C++

```
textBlock1->PointerEntered +=  
ref new PointerEventHandler(this, &BlankPage::textBlock1_PointerEntered);
```

Removing event handlers in code

It's not usually necessary to remove event handlers in code, even if you added them in code. The object lifetime behavior for most Windows Runtime objects such as pages and controls will destroy the objects when they are disconnected from the main [Window](#) and its visual tree, and any delegate references are destroyed too. .NET does this through garbage collection and Windows Runtime with C++/CX uses weak references by default.

There are some rare cases where you do want to remove event handlers explicitly. These include:

- Handlers you added for static events, which can't get garbage-collected in a conventional way. Examples of static events in the Windows Runtime API are the events of the [CompositionTarget](#) and [Clipboard](#) classes.
- Test code where you want the timing of handler removal to be immediate, or code where you want to swap old/new event handlers for an event at run time.
- The implementation of a custom **remove** accessor.
- Custom static events.
- Handlers for page navigations.

[FrameworkElement.Unloaded](#) or [Page.NavigatedFrom](#) are possible event triggers that have appropriate positions in state management and object lifetime such that you can use them for removing handlers for other events.

For example, you can remove an event handler named **textBlock1_PointerEntered** from the target object **textBlock1** using this code.

C#

```
textBlock1.PointerEntered -= textBlock1_PointerEntered;
```

VB

```
RemoveHandler textBlock1.PointerEntered, AddressOf textBlock1_PointerEntered
```

You can also remove handlers for cases where the event was added through a XAML attribute, which means that the handler was added in generated code. This is easier to do if you provided a **Name** value for the element where the handler was attached, because that provides an object reference for code later; however, you could also walk the object tree in order to find the necessary object reference in cases where the object has no **Name**.

If you need to remove an event handler in C++/CX, you'll need a registration token, which you should've received from the return value of the `+=` event handler registration. That's because the value you use for the right side of the `-=` deregistration in the C++/CX syntax is the token, not the method name. For C++/CX, you can't remove handlers that were added as a XAML attribute because the C++/CX generated code doesn't save a token.

Routed events

The Windows Runtime with C#, Microsoft Visual Basic or C++/CX supports the concept of a routed event for a set of events that are present on most UI elements. These events are for input and user interaction scenarios, and they are implemented on the [UIElement](#) base class. Here's a list of input events that are routed events:

- [BringIntoViewRequested](#)
- [CharacterReceived](#)
- [ContextCanceled](#)
- [ContextRequested](#)
- [DoubleTapped](#)
- [DragEnter](#)
- [DragLeave](#)
- [DragOver](#)
- [DragStarting](#)
- [Drop](#)
- [DropCompleted](#)
- [GettingFocus](#)

- [GotFocus](#)
- [Holding](#)
- [KeyDown](#)
- [KeyUp](#)
- [LosingFocus](#)
- [LostFocus](#)
- [ManipulationCompleted](#)
- [ManipulationDelta](#)
- [ManipulationInertiaStarting](#)
- [ManipulationStarted](#)
- [ManipulationStarting](#)
- [NoFocusCandidateFound](#)
- [PointerCanceled](#)
- [PointerCaptureLost](#)
- [PointerEntered](#)
- [PointerExited](#)
- [PointerMoved](#)
- [PointerPressed](#)
- [PointerReleased](#)
- [PointerWheelChanged](#)
- [PreviewKeyDown](#)
- [PreviewKeyUp](#)
- [PointerWheelChanged](#)
- [RightTapped](#)
- [Tapped](#)

A routed event is an event that is potentially passed on (*routed*) from a child object to each of its successive parent objects in an object tree. The XAML structure of your UI approximates this tree, with the root of that tree being the root element in XAML. The true object tree might vary somewhat from the XAML element nesting, because the object tree doesn't include XAML language features such as property element tags. You can conceive of the routed event as *bubbling* from any XAML object element child element that fires the event, toward the parent object element that contains it. The event and its event data can be handled on multiple objects along the event route. If no element has handlers, the route potentially keeps going until the root element is reached.

If you know Web technologies such as Dynamic HTML (DHTML) or HTML5, you might already be familiar with the *bubbling* event concept.

When a routed event bubbles through its event route, any attached event handlers all access a shared instance of event data. Therefore, if any of the event data is writeable by

a handler, any changes made to event data will be passed on to the next handler, and may no longer represent the original event data from the event. When an event has a routed event behavior, the reference documentation will include remarks or other notations about the routed behavior.

The **OriginalSource** property of **RoutedEventArgs**

When an event bubbles up an event route, *sender* is no longer the same object as the event-raising object. Instead, *sender* is the object where the handler that is being invoked is attached.

In some cases, *sender* is not interesting, and you are instead interested in info such as which of the possible child objects the pointer is over when a pointer event fired, or which object in a larger UI held focus when a user pressed a keyboard key. For these cases, you can use the value of the **OriginalSource** property. At all points on the route, **OriginalSource** reports the original object that fired the event, instead of the object where the handler is attached. However, for **UIElement** input events, that original object is often an object that is not immediately visible in the page-level UI definition XAML. Instead, that original source object might be a templated part of a control. For example, if the user hovers the pointer over the very edge of a **Button**, for most pointer events the **OriginalSource** is a **Border** template part in the **Template**, not the **Button** itself.

Tip Input event bubbling is especially useful if you are creating a templated control. Any control that has a template can have a new template applied by its consumer. The consumer that's trying to recreate a working template might unintentionally eliminate some event handling declared in the default template. You can still provide control-level event handling by attaching handlers as part of the **OnApplyTemplate** override in the class definition. Then you can catch the input events that bubble up to the control's root on instantiation.

The **Handled** property

Several event data classes for specific routed events contain a property named **Handled**. For examples, see **PointerRoutedEventArgs.Handled**, **KeyRoutedEventArgs.Handled**, **DragEventArgs.Handled**. In all cases **Handled** is a settable Boolean property.

Setting the **Handled** property to **true** influences the event system behavior. When **Handled** is **true**, the routing stops for most event handlers; the event doesn't continue along the route to notify other attached handlers of that particular event case. What "handled" means in the context of the event and how your app responds to it is up to you. Basically, **Handled** is a simple protocol that enables app code to state that an

occurrence of an event doesn't need to bubble to any containers, your app logic has taken care of what needs done. Conversely though, you do have to be careful that you aren't handling events that probably should bubble so that built-in system or control behaviors can act. For example, handling low-level events within the parts or items of a selection control can be detrimental. The selection control might be looking for input events to know that the selection should change.

Not all of the routed events can cancel a route in this way, and you can tell that because they won't have a **Handled** property. For example, [GotFocus](#) and [LostFocus](#) do bubble, but they always bubble all the way to the root, and their event data classes don't have a **Handled** property that can influence that behavior.

Input event handlers in controls

Specific Windows Runtime controls sometimes use the **Handled** concept for input events internally. This can make it seem like an input event never occurs, because your user code can't handle it. For example, the [Button](#) class includes logic that deliberately handles the general input event [PointerPressed](#). It does so because buttons fire a [Click](#) event that is initiated by pointer-pressed input, as well as by other input modes such as handling keys like the Enter key that can invoke the button when it's focused. For purposes of the class design of **Button**, the raw input event is conceptually handled, and class consumers such as your user code can instead interact with the control-relevant **Click** event. Topics for specific control classes in the Windows Runtime API reference often note the event handling behavior that the class implements. In some cases, you can change the behavior by overriding *OnEvent* methods. For example, you can change how your [TextBox](#) derived class reacts to key input by overriding [Control.OnKeyDown](#).

Registering handlers for already-handled routed events

Earlier we said that setting **Handled** to **true** prevents most handlers from being called. But the [AddHandler](#) method provides a technique where you can attach a handler that is always invoked for the route, even if some other handler earlier in the route has set **Handled** to **true** in the shared event data. This technique is useful if a control you are using has handled the event in its internal compositing or for control-specific logic, but you still want to respond to it from a control instance, or your app UI. But use this technique with caution, because it can contradict the purpose of **Handled** and possibly break a control's intended interactions.

Only the routed events that have a corresponding routed event identifier can use the [AddHandler](#) event handling technique, because the identifier is a required input of the [AddHandler](#) method. See the reference documentation for [AddHandler](#) for a list of events that have routed event identifiers available. For the most part this is the same list of routed events we showed you earlier. The exception is that the last two in the list: [GotFocus](#) and [LostFocus](#) don't have a routed event identifier, so you can't use [AddHandler](#) for those.

Routed events outside the visual tree

Certain objects participate in a relationship with the primary visual tree that is conceptually like having an overlay over the main visuals. These objects are not part of the usual parent-child relationships that connect all tree elements to the visual root. This is the case for any displayed [Popup](#) or [ToolTip](#). If you want to handle routed events from a [Popup](#) or [ToolTip](#), place the handlers on specific UI elements that are within the [Popup](#) or [ToolTip](#) and not the [Popup](#) or [ToolTip](#) elements themselves. Don't rely on routing inside any compositing that is performed for [Popup](#) or [ToolTip](#) content. This is because event routing for routed events works only along the main visual tree. A [Popup](#) or [ToolTip](#) is not considered a parent of subsidiary UI elements and never receives the routed event, even if it is trying to use something like the [Popup](#) default background as the capture area for input events.

Hit testing and input events

Determining whether and where in UI an element is visible to mouse, touch, and stylus input is called *hit testing*. For touch actions and also for interaction-specific or manipulation events that are consequences of a touch action, an element must be hit-test visible in order to be the event source and fire the event that is associated with the action. Otherwise, the action passes through the element to any underlying elements or parent elements in the visual tree that could interact with that input. There are several factors that affect hit testing, but you can determine whether a given element can fire input events by checking its [IsHitTestVisible](#) property. This property returns **true** only if the element meets these criteria:

- The element's [Visibility](#) property value is [Visible](#).
- The element's [Background](#) or [Fill](#) property value is not **null**. A **null** [Brush](#) value results in transparency and hit test invisibility. (To make an element transparent but also hit testable, use a [Transparent](#) brush instead of **null**.)

Note [Background](#) and [Fill](#) aren't defined by [UIElement](#), and are instead defined by different derived classes such as [Control](#) and [Shape](#). But the implications of brushes you

use for foreground and background properties are the same for hit testing and input events, no matter which subclass implements the properties.

- If the element is a control, its **IsEnabled** property value must be **true**.
- The element must have actual dimensions in layout. An element where either **ActualHeight** and **ActualWidth** are 0 won't fire input events.

Some controls have special rules for hit testing. For example, **TextBlock** has no **Background** property, but is still hit testable within the entire region of its dimensions. **Image** and **MediaElement** controls are hit testable over their defined rectangle dimensions, regardless of transparent content such as alpha channel in the media source file being displayed. **WebView** controls have special hit testing behavior because the input can be handled by the hosted HTML and fire script events.

Most **Panel** classes and **Border** are not hit-testable in their own background, but can still handle the user input events that are routed from the elements that they contain.

You can determine which elements are located at the same position as a user input event, regardless of whether the elements are hit-testable. To do this, call the **FindElementsInHostCoordinates** method. As the name implies, this method finds the elements at a location relative to a specified host element. However, applied transforms and layout changes can adjust the relative coordinate system of an element, and therefore affect which elements are found at a given location.

Commanding

A small number of UI elements support *commanding*. Commanding uses input-related routed events in its underlying implementation and enables processing of related UI input (a certain pointer action, a specific accelerator key) by invoking a single command handler. If commanding is available for a UI element, consider using its commanding APIs instead of any discrete input events. You typically use a **Binding** reference into properties of a class that defines the view model for data. The properties hold named commands that implement the language-specific **ICommand** commanding pattern. For more info, see **ButtonBase.Command**.

Custom events in the Windows Runtime

For purposes of defining custom events, how you add the event and what that means for your class design is highly dependent on which programming language you are using.

- For C# and Visual Basic, you are defining a CLR event. You can use the standard .NET event pattern, so long as you aren't using custom accessors (**add/remove**). Additional tips:
 - For the event handler it's a good idea to use [System.EventHandler<TEventArgs>](#) because it has built-in translation to the Windows Runtime generic event delegate [EventHandler<T>](#).
 - Don't base your event data class on [System.EventArgs](#) because it doesn't translate to the Windows Runtime. Use an existing event data class or no base class at all.
 - If you are using custom accessors, see [Custom events and event accessors in Windows Runtime components](#).
 - If you're not clear on what the standard .NET event pattern is, see [Defining Events for Custom Silverlight Classes](#). This is written for Microsoft Silverlight but it's still a good summation of the code and concepts for the standard .NET event pattern.
- For C++/CX, see [Events \(C++/CX\)](#).
 - Use named references even for your own usages of custom events. Don't use lambda for custom events, it can create a circular reference.

You can't declare a custom routed event for Windows Runtime; routed events are limited to the set that comes from the Windows Runtime.

Defining a custom event is usually done as part of the exercise of defining a custom control. It's a common pattern to have a dependency property that has a property-changed callback, and to also define a custom event that's fired by the dependency property callback in some or all cases. Consumers of your control don't have access to the property-changed callback you defined, but having a notification event available is the next best thing. For more info, see [Custom dependency properties](#).

Related topics

- [XAML overview](#)
- [Quickstart: Touch input](#)
- [Keyboard interactions](#)
- [.NET events and delegates](#)
- [Creating Windows Runtime components](#)
- [AddHandler](#)

Contacts, My People, and calendar

Article • 06/18/2024

You can let your users access their contacts and appointments so they can share content, email, calendar info, or messages with each other, or whatever functionality you design.

To see a few different ways in which your app can access contacts and appointments, see these topics:

 Expand table

Topic	Description
Select contacts	Through the Windows.ApplicationModel.Contacts namespace, you have several options for selecting contacts. Here, we'll show you how to select a single contact or multiple contacts, and we'll show you how to configure the contact picker to retrieve only the contact information that your app needs.
Send email	Shows how to launch the compose email dialog to allow the user to send an email message. You can pre-populate the fields of the email with data before showing the dialog. The message will not be sent until the user taps the send button.
Send an SMS message	This topic shows you how to launch the compose SMS dialog to allow the user to send an SMS message. You can pre-populate the fields of the SMS with data before showing the dialog. The message will not be sent until the user taps the send button.
Manage appointments	Through the Windows.ApplicationModel.Appointments namespace, you can create and manage appointments in a user's calendar app. Here, we'll show you how to create an appointment, add it to a calendar app, replace it in the calendar app, and remove it from the calendar app. We'll also show how to display a time span for a calendar app and create an appointment-recurrence object.
Connect your app to actions on a contact card	Shows how to make your app appear next to actions on a contact card or mini contact card. Users can choose your app to perform an action such as open a profile page, place a call, or send a message.
Adding My People support to an application	Shows how to add My People support to an application and how to pin and unpin contacts on the taskbar.
My People sharing	Shows how to add support for My People sharing, which lets users share content with their pinned contacts by dragging files from the File Explorer to a My People pin.

Topic	Description
My People notifications	Shows how to create and use My People notifications, a new kind of toast notification that's sent from a pinned contact.
Third party contacts for Windows People Experiences	Shows how third party packaged apps can integrate with Windows People Experiences by leveraging Windows APIs to store their contacts.

Related topics

- [Appointments API sample](#) 
- [Contact manager API sample](#) 
- [Contact Picker app sample](#) 
- [Handling Contact Actions sample](#) 
- [Contact Card Integration Sample](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Adding My People support to an application

Article • 01/04/2024

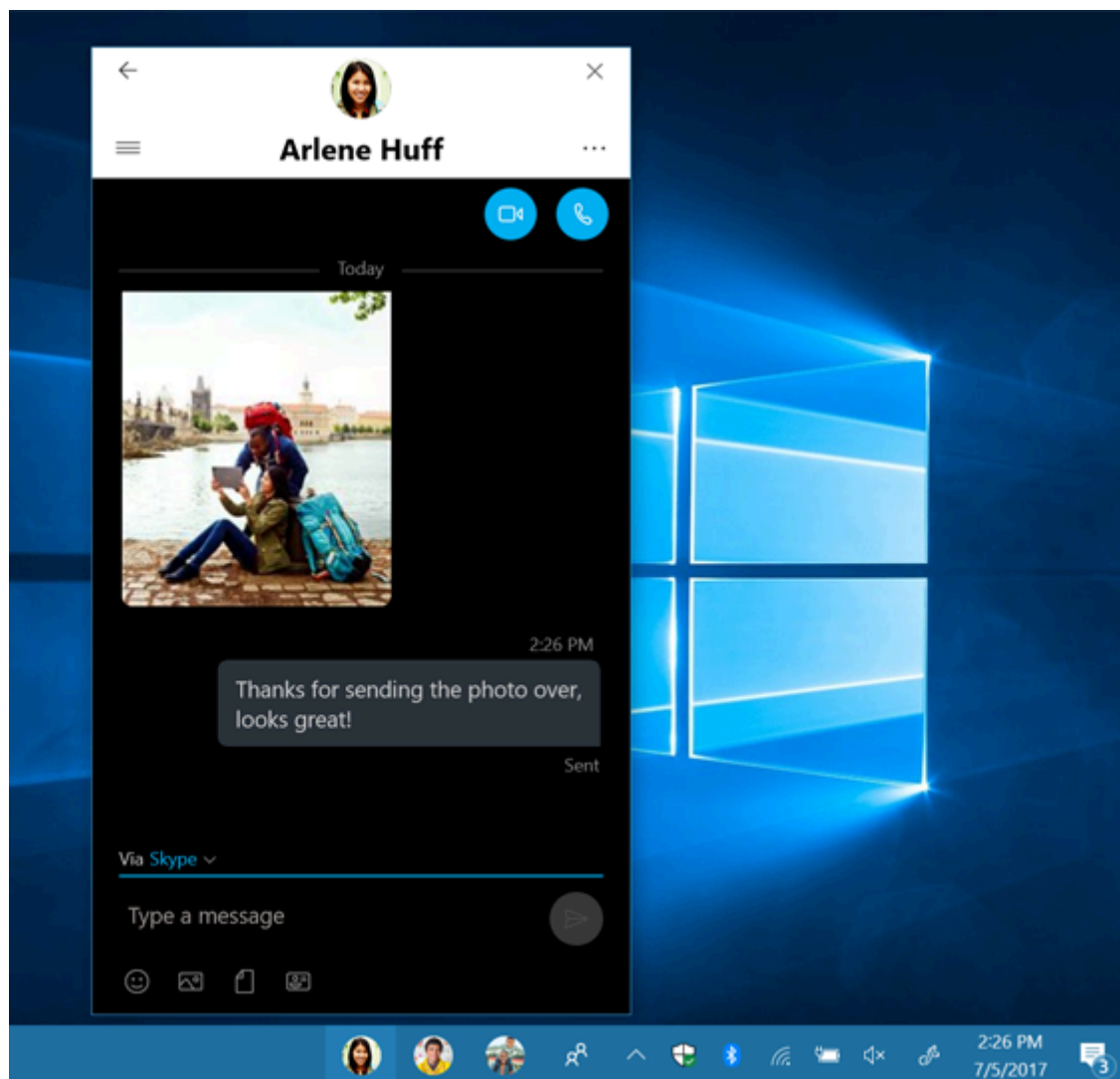
Important

My people is no longer supported in Windows 11 and Windows 10 versions with KB5034203 applied.

Note

As of the Windows 10 May 2019 Update (version 1903), new Windows 10 installations will no longer show 'People in the taskbar' by default. Customers can enable the feature by right-clicking on the taskbar and pressing "Show People on the taskbar." Developers are discouraged from adding My People support to their applications, and should visit the [Windows Developer Blog](#)  for more information about optimizing apps for Windows 10.

The My People feature allows users to pin contacts from an application directly to their taskbar, which creates a new contact object that they can interact with in several ways. This article shows how you can add support for this feature, allowing users to pin contacts directly from your app. When contacts are pinned, new types of user interaction become available, such as [My People sharing](#) and [notifications](#).



Requirements

- Windows 10 and Microsoft Visual Studio 2019. For installation details, see [Get set up with Visual Studio](#).
- Basic knowledge of C# or a similar object-oriented programming language. To get started with C#, see [Create a "Hello, world" app](#).

Overview

There are three things you need to do to enable your application to use the My People feature:

1. [Declare support for the shareTarget activation contract in your application manifest.](#)
2. [Annotate the contacts that the users can share to using your app.](#)
3. Support multiple instances of your application running at the same time. Users must be able to interact with a full version of your application while using it in a contact panel. They may even use it in multiple contact panels at once. To support

this, your application needs to be able to run multiple views simultaneously. To learn how to do this, see the article ["show multiple views for an app"](#).

When you've done this, your application will appear in the contact panel for annotated contacts.

Declaring support for the contract

To declare support for the My People contract, open your application in Visual Studio. From the **Solution Explorer**, right click **Package.appxmanifest** and select **Open With**. From the menu, select **XML (Text) Editor** and click **OK**. Make the following changes to the manifest:

Before

XML

```
<Package
  xmlns="http://schemas.microsoft.com/appx/manifest/foundation/windows10"
  xmlns:uap="http://schemas.microsoft.com/appx/manifest/uap/windows10" >

  <Applications>
    <Application Id="MyApp"
      Executable="$targetnametoken$.exe"
      EntryPoint="My.App">
    </Application>
  </Applications>
```

After

XML

```
<Package
  xmlns="http://schemas.microsoft.com/appx/manifest/foundation/windows10"
  xmlns:uap="http://schemas.microsoft.com/appx/manifest/uap/windows10"
  xmlns:uap4="http://schemas.microsoft.com/appx/manifest/uap/windows10/4" >

  <Applications>
    <Application Id="MyApp"
      Executable="$targetnametoken$.exe"
      EntryPoint="My.App">
      <Extensions>
        <uap4:Extension Category="windows.contactPanel" />
      </Extensions>
    </Application>
  </Applications>
```

With this addition, your application can now be launched through the `windows.ContactPanel` contract, which allows you to interact with contact panels.

Annotating contacts

To allow contacts from your application to appear in the taskbar via the My People pane, you need to write them to the Windows contact store. To learn how to write contacts, see the [Contact Card sample](#).

Your application must also write an annotation to each contact. Annotations are pieces of data from your application that are associated with a contact. The annotation must contain the activatable class corresponding to your desired view in its **ProviderProperties** member, and declare support for the **ContactProfile** operation.

You can annotate contacts at any point while your app is running, but generally you should annotate contacts as soon as they are added to the Windows contact store.

Csharp

```
if
(ApiInformation.IsApiContractPresent("Windows.Foundation.UniversalApiContrac
t", 5))
{
    // Create a new contact annotation
    ContactAnnotation annotation = new ContactAnnotation();
    annotation.ContactId = myContact.Id;

    // Add appId and contact panel support to the annotation
    String appId = "MyApp_vqvv5s4y3scbg!App";
    annotation.ProviderProperties.Add("ContactPanelAppID", appId);
    annotation.SupportedOperations =
ContactAnnotationOperations.ContactProfile;

    // Save annotation to contact annotation list
    // Windows.ApplicationModel.Contacts.ContactAnnotationList
    await contactAnnotationList.TrySaveAnnotationAsync(annotation));
}
```

The “appid” is the Package Family Name, followed by ‘!’ and the activatable class ID. To find your Package Family Name, open **Package.appxmanifest** using the default editor, and look in the “Packaging” tab. Here, “App” is the activatable class corresponding to the application startup view.

Allow contacts to invite new potential users

By default, your application will only appear in the contact panel for contacts that you have specifically annotated. This is to avoid confusion with contacts that can't be interacted with through your app. If you want your application to appear for contacts that your application doesn't know about (to invite users to add that contact to their account, for instance), you can add the following to your manifest:

Before

Csharp

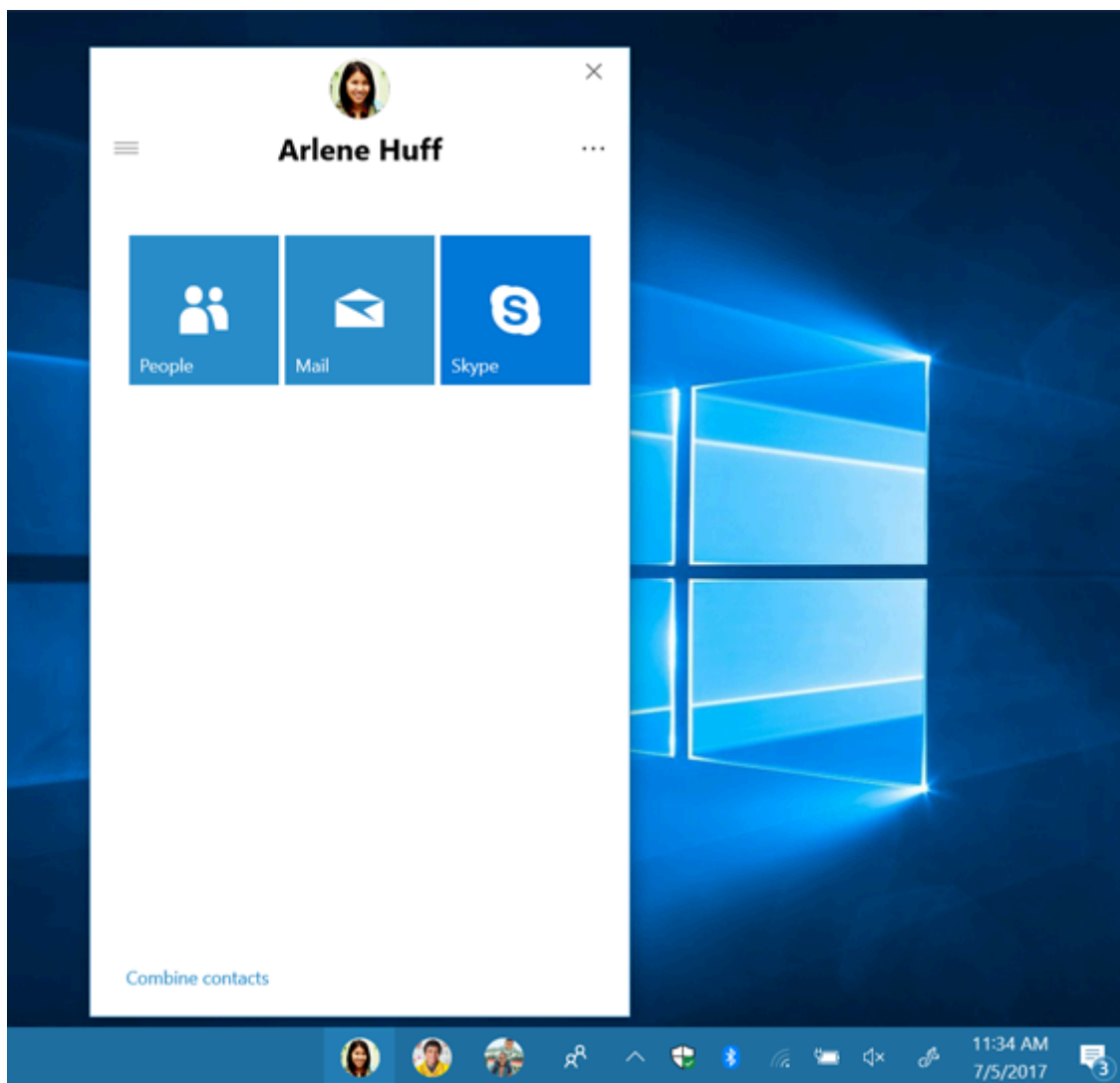
```
<Applications>
  <Application Id="MyApp"
    Executable="$targetnametoken$.exe"
    EntryPoint="My.App">
    <Extensions>
      <uap4:Extension Category="windows.contactPanel" />
    </Extensions>
  </Application>
</Applications>
```

After

Csharp

```
<Applications>
  <Application Id="MyApp"
    Executable="$targetnametoken$.exe"
    EntryPoint="My.App">
    <Extensions>
      <uap4:Extension Category="windows.contactPanel">
        <uap4:ContactPanel SupportsUnknownContacts="true" />
      </uap4:Extension>
    </Extensions>
  </Application>
</Applications>
```

With this change, your application will appear as an available option in the contact panel for all contacts that the user has pinned. When your application is activated using the contact panel contract, you should check to see if the contact is one your application knows about. If not, you should show your app's new user experience.



Support for email apps

If you are writing an email app, you don't need to annotate every contact manually. If you declare support for the contact pane and for the `mailto:` protocol, your application will automatically appear for users with an email address.

Running in the contact panel

Now that your application appears in the contact panel for some or all users, you need to handle activation with the contact panel contract.

Csharp

```
override protected void OnActivated(IActivatedEventArgs e)
{
    if (e.Kind == ActivationKind.ContactPanel)
    {
        // Create a Frame to act as the navigation context and navigate to
        the first page
        var rootFrame = new Frame();
```

```

// Place the frame in the current Window
Window.Current.Content = rootFrame;

// Navigate to the page that shows the Contact UI.
rootFrame.Navigate(typeof(ContactPage), e);

// Ensure the current window is active
Window.Current.Activate();
    }
}

```

When your application is activated with this contract, it will receive a [ContactPanelActivatedEventArgs](#) object. This contains the ID of the Contact that your application is trying to interact with on launch, and a [ContactPanel](#) object. You should keep a reference to this [ContactPanel](#) object, which will allow you to interact with the panel.

The [ContactPanel](#) object has two events your application should listen for:

- The **LaunchFullAppRequested** event is sent when the user has invoked the UI element that requests that your full application be launched in its own window. Your application is responsible for launching itself, passing along all necessary context. You are free to do this however you'd like (for example, via protocol launch).
- The **Closing** event is sent when your application is about to be closed, allowing you to save any context.

The [ContactPanel](#) object also allows you to set the background color of the contact panel header (if not set, it will default to the system theme) and to programmatically close the contact panel.

Supporting notification badging

If you want contacts pinned to the taskbar to be badged when new notifications arrive from your app that are related to that person, then you must include the **hint-people** parameter in your [toast notifications](#) and expressive [My People notifications](#).



To badge a contact, the top-level toast node must include the hint-people parameter to indicate the sending or related contact. This parameter can have any of the following values:

- **Email address**
 - E.g. johndoe@mydomain.com
- **Telephone number**
 - E.g. [888-888-8888](tel:888-888-8888)
- **Remote ID**
 - E.g. remoteid:1234

Here is an example of how to identify a toast notification is related to a specific person:

XML

```
<toast hint-people="mailto:johndoe@mydomain.com">
  <visual lang="en-US">
    <binding template="ToastText01">
      <text>John Doe posted a comment.</text>
    </binding>
  </visual>
</toast>
```

ⓘ Note

If your app uses the [ContactStore APIs](#) and uses the [StoredContact.RemoteId](#) property to link contacts stored on the PC with contacts stored remotely, it is essential that the value for the RemoteId property is both stable and unique. This means that the remote ID must consistently identify a single user account and should contain a unique tag to guarantee that it does not conflict with the remote IDs of other contacts on the PC, including contacts that are owned by other apps. If the remote IDs used by your app are not guaranteed to be stable and unique, you can use the RemoteIdHelper class shown later in this topic in order to add a unique tag to all of your remote IDs before you add them to the system. Or you can choose to not use the RemoteId property at all and instead you create a custom extended property in which to store remote IDs for your contacts.

The PinnedContactManager class

The [PinnedContactManager](#) is used to manage which contacts are pinned to the taskbar. This class lets you pin and unpin contacts, determine whether a contact is

pinned, and determine if pinning on a particular surface is supported by the system your application is currently running on.

You can retrieve the `PinnedContactManager` object using the **GetDefault** method:

Csharp

```
PinnedContactManager pinnedContactManager =  
PinnedContactManager.GetDefault();
```

Pinning and unpinning contacts

You can now pin and unpin contacts using the `PinnedContactManager` you just created. The **RequestPinContactAsync** and **RequestUnpinContactAsync** methods provide the user with confirmation dialogs, so they must be called from your Application Single-Threaded Apartment (ASTA, or UI) thread.

Csharp

```
async void PinContact (Contact contact)  
{  
    await pinnedContactManager.RequestPinContactAsync(contact,  
  
PinnedContactSurface.Taskbar);  
}  
  
async void UnpinContact (Contact contact)  
{  
    await pinnedContactManager.RequestUnpinContactAsync(contact,  
  
PinnedContactSurface.Taskbar);  
}
```

You can also pin multiple contacts at the same time:

Csharp

```
async Task PinMultipleContacts(Contact[] contacts)  
{  
    await pinnedContactManager.RequestPinContactsAsync(  
        contacts, PinnedContactSurface.Taskbar);  
}
```

ⓘ Note

There is currently no batch operation for unpinning contacts.

Note:

See also

- [My People sharing](#)
- [My People notifications](#)
- [My People integration sample](#) 
- [Contact Card sample](#) 
- [PinnedContactManager class documentation](#)
- [Connect your app to actions on a contact card](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

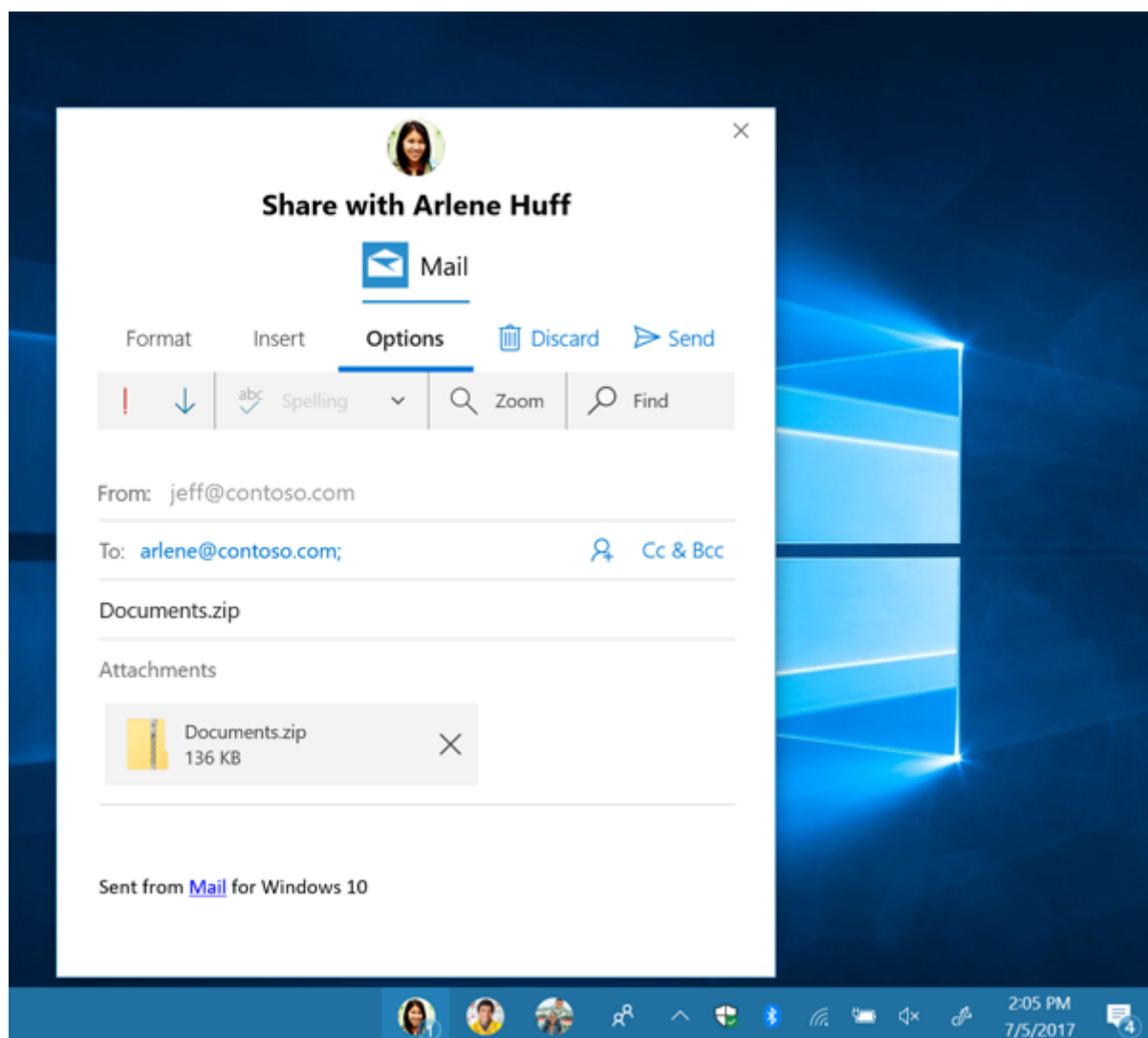
My People sharing

Article • 01/04/2024

❗ Important

My people is no longer supported in Windows 11 and Windows 10 versions with KB5034203 applied.

The My People feature allows users to pin contacts to their taskbar, enabling them to stay in touch easily from anywhere in Windows, no matter what application they are connected by. Now users can share content with their pinned contacts by dragging files from the File Explorer to their My People pin. They can also share to any contacts in the Windows contact store via the standard share charm. Keep reading to learn how to enable your application as a My People sharing target.



Requirements

- Windows 10 and Microsoft Visual Studio 2019. For installation details, see [Get set up with Visual Studio](#).
- Basic knowledge of C# or a similar object-oriented programming language. To get started with C#, see [Create a "Hello, world" app](#).

Overview

There are three steps you must take to enable your application as a My People sharing target:

1. [Declare support for the shareTarget activation contract in your application manifest](#).
2. [Annotate the contacts that the users can share to using your app](#).
3. Support multiple instances of the application running at the same time. Users must be able to interact with a full version of your application while also using it to share with others. They may use it in multiple share windows at once. To support this, your application needs to be able to run multiple views simultaneously. To learn how to do this, see the article ["show multiple views for an app"](#).

When you've done this, your application will appear as a share target in the My People share window, which can be launched in two ways:

1. A contact is chosen via the share charm.
2. File(s) are dragged and dropped on a contact pinned to the taskbar.

Declaring support for the share contract

To declare support for your application as a share target, first open your application in Visual Studio. From the **Solution Explorer**, right click **Package.appxmanifest** and select **Open With**. From the menu, select **XML (Text) Editor** and click **OK**. Then, make the following changes to the manifest:

Before

XML

```
<Applications>
  <Application Id="MyApp"
    Executable="$targetnametoken$.exe"
    EntryPoint="My.App">
  </Application>
</Applications>
```


After

XML

```
<Applications>
  <Application Id="MyApp"
    Executable="$targetnametoken$.exe"
    EntryPoint="My.App">
    <Extensions>
      <uap:Extension Category="windows.shareTarget">
        <uap:ShareTarget Description="Share with MyApp">
          <uap:SupportedFileTypes>
            <uap:SupportsAnyFileType/>
          </uap:SupportedFileTypes>
          <uap:DataFormat>Text</uap:DataFormat>
          <uap:DataFormat>Bitmap</uap:DataFormat>
          <uap:DataFormat>Html</uap:DataFormat>
          <uap:DataFormat>StorageItems</uap:DataFormat>
          <uap:DataFormat>URI</uap:DataFormat>
        </uap:ShareTarget>
      </uap:Extension>
    </Extensions>
  </Application>
</Applications>
```

This code adds support for all files and data formats, but you can choose to specify what files types and data formats are supported (see [ShareTarget class documentation](#) for more details).

Annotating contacts

To allow the My People share window to show your application as a share target for your contacts, you need to write them to the Windows contact store. To learn how to write contacts, see the [Contact Card Integration sample](#) [↗](#).

For your application to appear as a My People share target when sharing to a contact, it must write an annotation to that contact. Annotations are pieces of data from your application that are associated with a contact. The annotation must contain the activatable class corresponding to your desired view in its **ProviderProperties** member, and declare support for the **Share** operation.

You can annotate contacts at any point while your app is running, but generally you should annotate contacts as soon as they are added to the Windows contact store.

Csharp

```

if
(ApiInformation.IsApiContractPresent("Windows.Foundation.UniversalApiContrac
t", 5))
{
    // Create a new contact annotation
    ContactAnnotation annotation = new ContactAnnotation();
    annotation.ContactId = myContact.Id;

    // Add appId and Share support to the annotation
    String appId = "MyApp_vqv5s4y3scbg!App";
    annotation.ProviderProperties.Add("ContactShareAppID", appId);
    annotation.SupportedOperations = ContactAnnotationOperations::Share;

    // Save annotation to contact annotation list
    // Windows.ApplicationModel.Contacts.ContactAnnotationList
    await contactAnnotationList.TrySaveAnnotationAsync(annotation);
}

```

The “appId” is the Package Family Name, followed by ‘!’ and the Activatable Class ID. To find your Package Family Name, open **Package.appxmanifest** using the default editor, and look in the “Packaging” tab. Here, “App” is the Activatable Class corresponding to the Share Target view.

Running as a My People share target

Finally, to run the app, override the [OnShareTargetActivated](#) method in your app’s main class to handle the share target activation. The [ShareTargetActivatedEventArgs.ShareOperation.Contacts](#) property will contain the contact(s) that are being shared to, or will be empty if this is a standard share operation (not a My People share).

```

Csharp

protected override void OnShareTargetActivated(ShareTargetActivatedEventArgs
args)
{
    bool isPeopleShare = false;
    if
(ApiInformation.IsApiContractPresent("Windows.Foundation.UniversalApiContrac
t", 5))
    {
        // Make sure the current OS version includes the My People feature
        before
        // accessing the ShareOperation.Contacts property
        isPeopleShare = (args.ShareOperation.Contacts.Count > 0);
    }

    if (isPeopleShare)

```

```
{  
    // Show share UI for MyPeople contact(s)  
}  
else  
{  
    // Show standard share UI for unpinned contacts  
}  
}
```

See also

- [Adding My People support](#)
- [ShareTarget Class](#)
- [Contact Card Integration sample](#) ↗

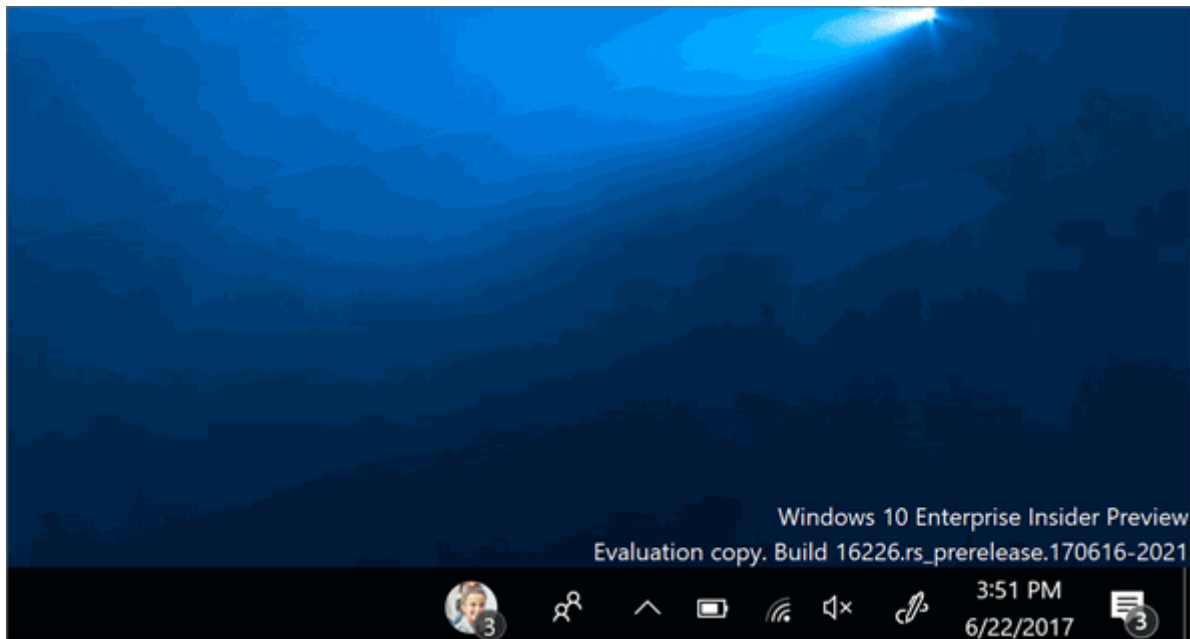
My People notifications

Article • 01/04/2024

❗ Important

My people is no longer supported in Windows 11 and Windows 10 versions with KB5034203 applied.

My People notifications provide a new way for users to connect with the people they care about, through quick expressive gestures. This article shows how to design and implement My People notifications in your application. For complete implementations, see the [My People Notifications Sample](#).[↗]



Requirements

- Windows 10 and Microsoft Visual Studio 2019 or later. For installation details, see [Get set up with Visual Studio](#).
- Basic knowledge of C# or a similar object-oriented programming language. To get started with C#, see [Create a "Hello, world" app](#).

How it works

As an alternative to generic toast notifications, you can now send notifications through the My People feature to provide a more personal experience to users. This is a new kind of toast, sent from a contact pinned on the user's taskbar with the My People

feature. When the notification is received, the sender's contact picture will animate in the taskbar and a sound will play, signaling that the notification is starting. The animation or image specified in the payload will be displayed for 5 seconds (or, if the payload is an animation less than 5 seconds long, it will loop until 5 seconds have passed).

Supported image types

- GIF
- Static Image (JPEG, PNG)
- Spritesheet (vertical only)

ⓘ Note

A spritesheet is an animation derived from a static image (JPEG or PNG). Individual frames are arranged vertically, such that the first frame is on top (though you can specify a different starting frame in the toast payload). Each frame must have the same height, which the program loops through to create an animated sequence (like a flipbook with its pages laid out vertically). An example of a spritesheet is shown below.



Notification parameters

My People notifications use the [toast notification](#) framework, but require an additional binding node in the toast payload. This second binding must include the following parameter:

XML

```
experienceType="shoulderTap"
```

This indicates that the toast should be treated as a My People notification.

The image node inside the binding should include the following parameters:

- **src**

- The URI of the asset. This can be either HTTP/HTTPS web URI, an msappx URI, or a path to a local file.
- **spritesheet-src**
 - The URI of the asset. This can be either HTTP/HTTPS web URI, an msappx URI, or a path to a local file. Only required for spritesheet animations.
- **spritesheet-height**
 - The frame height (in pixels). Only required for spritesheet animations.
- **spritesheet-fps**
 - Frames per second (FPS). Only required for spritesheet animations. Only values 1-120 are supported.
- **spritesheet-startingFrame**
 - The frame number to begin the animation. Only used for spritesheet animations and defaults to 0 if not provided.
- **alt**
 - Text string used for screen reader narration.

ⓘ Note

When making an animated notification, you should still specify a static image in the "src" parameter. It will be used as a fall-back if the animation fails to display.

In addition, the top-level toast node must include the **hint-people** parameter to specify the sending contact. This parameter can have any the following values:

- **Email address**
 - E.g. `mailto:johndoe@mydomain.com`
- **Telephone number**
 - E.g. `888-888-8888`
- **Remote ID**
 - E.g. `remoteid:1234`

ⓘ Note

If your app uses the **ContactStore** APIs and uses the **StoredContact.RemoteId** property to link contacts stored on the PC with contacts stored remotely, it is essential that the value for the RemoteId property is both stable and unique. This means that the remote ID must consistently identify a single user account, and should contain a unique tag to guarantee that it does not conflict with the remote IDs of other contacts on the PC, including contacts that are owned by other apps. If the remote IDs used by your app are not guaranteed to be stable and unique, you can use the **RemoteIdHelper** class in order to add a unique tag to all of your

remote IDs before you add them to the system. Alternatively, you can choose to not use the `Remoteld` property at all, and instead create a custom extended property in which to store remote IDs for your contacts.

In addition to the second binding and payload, you must include another payload in the first binding for the fallback toast. The notification will use this if it is forced to revert to a regular toast (explained further at the [end of this article](#)).

Creating the notification

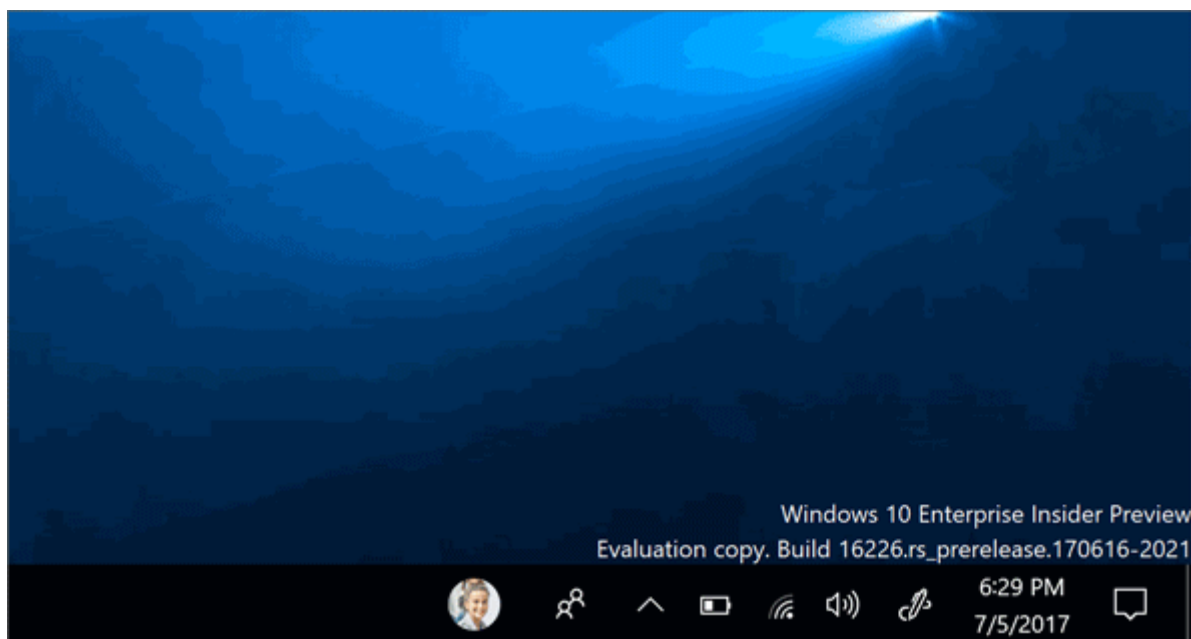
You can create a My People notification template just like you would a [toast notification](#).

Here's an example of how to create a My People notification with a static image payload:

XML

```
<toast hint-people="mailto:johndoe@mydomain.com">
  <visual lang="en-US">
    <binding template="ToastGeneric">
      <text hint-style="body">Toast fallback</text>
      <text>Add your fallback toast content here</text>
    </binding>
    <binding template="ToastGeneric" experienceType="shoulderTap">
      <image src="https://learn.microsoft.com/windows/uwp/contacts-and-calendar/images/shoulder-tap-static-payload.png"/>
    </binding>
  </visual>
</toast>
```

When you start the notification, it should look like this:

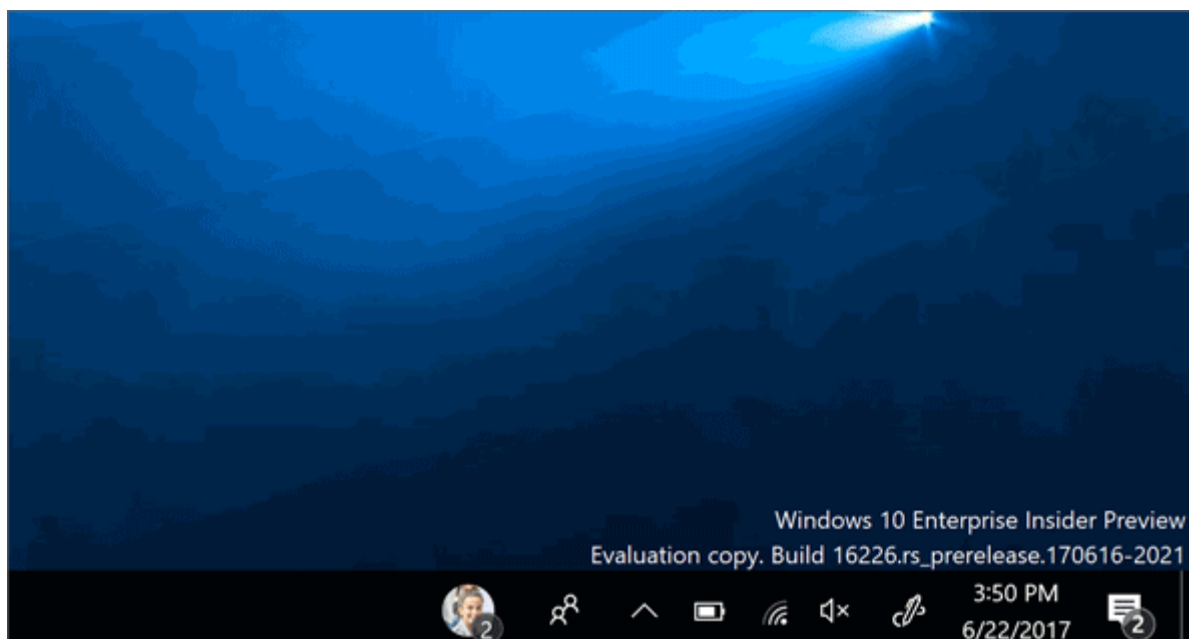


Here's an example of how to create a notification with an animated spritesheet payload. This spritesheet has a frame-height of 80 pixels, which we'll animate at 25 frames per second. We set the starting frame to 15 and provide it with a static fallback image in the "src" parameter. The fallback image is used if the spritesheet animation fails to display.

XML

```
<toast hint-people="mailto: johndoe@mydomain.com">
  <visual lang="en-US">
    <binding template="ToastGeneric">
      <text hint-style="body">Toast fallback</text>
      <text>Add your fallback toast content here</text>
    </binding>
    <binding template="ToastGeneric" experienceType="shoulderTap">
      <image src="https://learn.microsoft.com/windows/uwp/contacts-
and-calendar/images/shoulder-tap-pizza-static.png"
spritesheet-
src="https://learn.microsoft.com/windows/uwp/contacts-and-
calendar/images/shoulder-tap-pizza-spritesheet.png"
spritesheet-height='80' spritesheet-fps='25' spritesheet-
startingFrame='15' />
    </binding>
  </visual>
</toast>
```

When you start the notification, it should look like this:



Starting the notification

To start a My People notification, we need to convert the toast template into an [XmlDocument](#) object. When you have defined the toast in an XML file (here named "content.xml"), you can use this code to start it:

CSharp

```
string xmlText = File.ReadAllText("content.xml");  
XmlDocument xmlContent = new XmlDocument();  
xmlContent.LoadXml(xmlText);
```

You can then use this code to create and send the toast:

CSharp

```
ToastNotification notification = new ToastNotification(xmlContent);  
ToastNotificationManager.CreateToastNotifier().Show(notification);
```

Falling back to toast

There are some cases when a My People notification will instead display as a regular toast notification. A My People notification will fall back to toast under the following conditions:

- The notification fails to display
- My People notifications are not enabled by the recipient
- The sender's contact is not pinned to the receiver's taskbar

If a My People notification falls back to toast, the second My-People-specific binding is ignored, and only the first binding is used to display the toast. This is why it is critical to provide a fallback payload in the first toast binding.

See also

- [My People Notifications Sample](#) 
- [Adding My People support](#)
- [Adaptive toast notifications](#)
- [ToastNotification Class](#)

Select contacts

Article • 10/20/2022

Through the [Windows.ApplicationModel.Contacts](#) namespace, you have several options for selecting contacts. Here, we'll show you how to select a single contact or multiple contacts, and we'll show you how to configure the contact picker to retrieve only the contact information that your app needs.

Set up the contact picker

Create an instance of [Windows.ApplicationModel.Contacts.ContactPicker](#) and assign it to a variable.

C#

```
var contactPicker = new Windows.ApplicationModel.Contacts.ContactPicker();
```

Set the selection mode (optional)

By default, the contact picker retrieves all of the available data for the contacts that the user selects. The [SelectionMode](#) property lets you configure the contact picker to retrieve only the data fields that your app needs. This is a more efficient way to use the contact picker if you only need a subset of the available contact data.

First, set the [SelectionMode](#) property to **Fields**:

C#

```
contactPicker.SelectionMode =  
Windows.ApplicationModel.Contacts.ContactSelectionMode.Fields;
```

Then, use the [DesiredFieldsWithContactFieldType](#) property to specify the fields that you want the contact picker to retrieve. This example configures the contact picker to retrieve email addresses:

C#

```
contactPicker.DesiredFieldsWithContactFieldType.Add(Windows.ApplicationModel.  
.Contacts.ContactFieldType.Email);
```

Launch the picker

C#

```
Contact contact = await contactPicker.PickContactAsync();
```

Use [PickContactsAsync](#) if you want the user to select one or more contacts.

C#

```
public IList<Contact> contacts;  
contacts = await contactPicker.PickContactsAsync();
```

Process the contacts

When the picker returns, check whether the user has selected any contacts. If so, process the contact information.

This example shows how to process a single contact. Here we retrieve the contact's name and copy it into a [TextBlock](#) control called *OutputName*.

C#

```
if (contact != null)  
{  
    OutputName.Text = contact.DisplayName;  
}  
else  
{  
    rootPage.NotifyUser("No contact was selected.",  
        NotifyType.ErrorMessage);  
}
```

This example shows how to process multiple contacts.

C#

```
if (contacts != null && contacts.Count > 0)  
{  
    foreach (Contact contact in contacts)  
    {  
        // Do something with the contact information.  
    }  
}
```

Complete example (single contact)

This example uses the contact picker to retrieve a single contact's name along with an email address, location or phone number.

C#

```
// ...
using Windows.ApplicationModel.Contacts;
// ...

private async void PickAContactButton_Click(object sender, RoutedEventArgs e)
{
    ContactPicker contactPicker = new ContactPicker();

    contactPicker.DesiredFieldsWithContactFieldType.Add(ContactFieldType.Email);

    contactPicker.DesiredFieldsWithContactFieldType.Add(ContactFieldType.Address);

    contactPicker.DesiredFieldsWithContactFieldType.Add(ContactFieldType.PhoneNumber);

    Contact contact = await contactPicker.PickContactAsync();

    if (contact != null)
    {
        OutputFields.Visibility = Visibility.Visible;
        OutputEmpty.Visibility = Visibility.Collapsed;

        OutputName.Text = contact.DisplayName;

        AppendContactFieldValues(OutputEmails, contact.Emails);
        AppendContactFieldValues(OutputPhoneNumbers, contact.Phones);
        AppendContactFieldValues(OutputAddresses, contact.Addresses);
    }
    else
    {
        OutputEmpty.Visibility = Visibility.Visible;
        OutputFields.Visibility = Visibility.Collapsed;
    }
}

private void AppendContactFieldValues<T>(TextBlock content, IList<T> fields)
{
    if (fields.Count > 0)
    {
        StringBuilder output = new StringBuilder();

        if (fields[0].GetType() == typeof(ContactEmail))
        {
```

```

        foreach (ContactEmail email in fields as IList<ContactEmail>)
        {
            output.AppendFormat("Email: {0} ({1})\n", email.Address,
email.Kind);
        }
    }
    else if (fields[0].GetType() == typeof(ContactPhone))
    {
        foreach (ContactPhone phone in fields as IList<ContactPhone>)
        {
            output.AppendFormat("Phone: {0} ({1})\n", phone.Number,
phone.Kind);
        }
    }
    else if (fields[0].GetType() == typeof(ContactAddress))
    {
        List<String> addressParts = null;
        string unstructuredAddress = "";

        foreach (ContactAddress address in fields as
IList<ContactAddress>)
        {
            addressParts = (new List<string> { address.StreetAddress,
address.Locality, address.Region, address.PostalCode });
            unstructuredAddress = string.Join(", ",
addressParts.FindAll(s => !string.IsNullOrEmpty(s)));
            output.AppendFormat("Address: {0} ({1})\n",
unstructuredAddress, address.Kind);
        }
    }

    content.Visibility = Visibility.Visible;
    content.Text = output.ToString();
}
else
{
    content.Visibility = Visibility.Collapsed;
}
}

```

Complete example (multiple contacts)

This example uses the contact picker to retrieve multiple contacts and then adds the contacts to a [ListView](#) control called `OutputContacts`.

C#

```

MainPage rootPage = MainPage.Current;
public IList<Contact> contacts;

private async void PickContactsButton_Click(object sender, RoutedEventArgs

```

```

e)
{
    var contactPicker = new
Windows.ApplicationModel.Contacts.ContactPicker();
    contactPicker.CommitButtonText = "Select";
    contacts = await contactPicker.PickContactsAsync();

    // Clear the ListView.
    OutputContacts.Items.Clear();

    if (contacts != null && contacts.Count > 0)
    {
        OutputContacts.Visibility = Windows.UI.Xaml.Visibility.Visible;
        OutputEmpty.Visibility = Visibility.Collapsed;

        foreach (Contact contact in contacts)
        {
            // Add the contacts to the ListView.
            OutputContacts.Items.Add(new ContactItemAdapter(contact));
        }
    }
    else
    {
        OutputEmpty.Visibility = Visibility.Visible;
    }
}

```

C#

```

public class ContactItemAdapter
{
    public string Name { get; private set; }
    public string SecondaryText { get; private set; }

    public ContactItemAdapter(Contact contact)
    {
        Name = contact.DisplayName;
        if (contact.Emails.Count > 0)
        {
            SecondaryText = contact.Emails[0].Address;
        }
        else if (contact.Phones.Count > 0)
        {
            SecondaryText = contact.Phones[0].Number;
        }
        else if (contact.Addresses.Count > 0)
        {
            List<string> addressParts = (new List<string> {
contact.Addresses[0].StreetAddress,
            contact.Addresses[0].Locality, contact.Addresses[0].Region,
contact.Addresses[0].PostalCode });
            string unstructuredAddress = string.Join(", ",
addressParts.FindAll(s => !string.IsNullOrEmpty(s)));

```



```
        SecondaryText = unstructuredAddress;  
    }  
}  
}
```

Summary and next steps

Now you have a basic understanding of how to use the contact picker to retrieve contact information. Download the [Universal Windows app samples](#) from GitHub to see more examples of how to use contacts and the contact picker.

Send email

Article • 10/20/2022

Shows how to launch the compose email dialog to allow the user to send an email message. You can pre-populate the fields of the email with data before showing the dialog. The message will not be sent until the user taps the send button.

In this article

- [Launch the compose email dialog](#)
- [Summary and next steps](#)
- [Related topics](#)

Launch the compose email dialog

Create a new [EmailMessage](#) object and set the data that you want to be pre-populated in the compose email dialog. Call [ShowComposeNewEmailAsync](#) to show the dialog.

C#

```
private async Task ComposeEmail(Windows.ApplicationModel.Contacts.Contact
recipient,
    string subject, string messageBody)
{
    var emailMessage = new Windows.ApplicationModel.Email.EmailMessage();
    emailMessage.Body = messageBody;

    var email =
recipient.Emails.FirstOrDefault<Windows.ApplicationModel.Contacts.ContactEmail>();
    if (email != null)
    {
        var emailRecipient = new
Windows.ApplicationModel.Email.EmailRecipient(email.Address);
        emailMessage.To.Add(emailRecipient);
        emailMessage.Subject = subject;
    }

    await
Windows.ApplicationModel.Email.EmailManager.ShowComposeNewEmailAsync(emailMessage);
}
```

ⓘ Note

Attachments that you add to an email by using the **EmailAttachment** class will appear only in the Mail app. If users have any other mail program configured as their default mail program, the compose window will appear without the attachment. This is a known issue.

Summary and next steps

This topic has shown you how to launch the compose email dialog. For information on selecting contacts to use as recipients for an email message, see [Select contacts](#). See [PickSingleFileAsync](#) to select a file to use as an email attachment.

Related topics

- [Selecting contacts](#)

Send an SMS message

Article • 01/04/2023

This topic shows you how to launch the compose SMS dialog to allow the user to send an SMS message. You can pre-populate the fields of the SMS with data before showing the dialog. The message will not be sent until the user taps the send button.

To call this code, declare the **chat**, **smsSend**, and **chatSystem** capabilities in your package manifest. These are [restricted capabilities](#) but you can use them in your app. You need approval only if you intend to publish your app to the Store. See [Account types, locations, and fees](#).

Launch the compose SMS dialog

Create a new [ChatMessage](#) object and set the data that you want to be pre-populated in the compose email dialog. Call [ShowComposeSmsMessageAsync](#) to show the dialog.

C#

```
private async void ComposeSms(Windows.ApplicationModel.Contacts.Contact
recipient,
    string messageBody,
    StorageFile attachmentFile,
    string mimeType)
{
    var chatMessage = new Windows.ApplicationModel.Chat.ChatMessage();
    chatMessage.Body = messageBody;

    if (attachmentFile != null)
    {
        var stream =
Windows.Storage.Streams.RandomAccessStreamReference.CreateFromFile(attachmen
tFile);

        var attachment = new
Windows.ApplicationModel.Chat.ChatMessageAttachment(
            mimeType,
            stream);

        chatMessage.Attachments.Add(attachment);
    }

    var phone =
recipient.Phones.FirstOrDefault<Windows.ApplicationModel.Contacts.ContactPho
ne>();
    if (phone != null)
    {
```

```
        chatMessage.Recipients.Add(phone.Number);  
    }  
    await  
    Windows.ApplicationModel.Chat.ChatMessageManager.ShowComposeSmsMessageAsync(  
        chatMessage);  
}
```

You can use the following code to determine whether the device that is running your app is able to send SMS messages.

```
C#  
  
if  
(Windows.Foundation.Metadata.ApiInformation.IsTypePresent("Windows.Applicati  
onModel.Chat"))  
{  
    // Call code here.  
}
```

Summary and next steps

This topic has shown you how to launch the compose SMS dialog. For information on selecting contacts to use as recipients for an SMS message, see [Select contacts](#).

Download the [Universal Windows app samples](#) [↗](#) from GitHub to see more examples of how to send and receive SMS messages by using a background task.

Related topics

- [Select contacts](#)

Manage appointments

Article • 10/20/2022

Through the [Windows.ApplicationModel.Appointments](#) namespace, you can create and manage appointments in a user's calendar app. Here, we'll show you how to create an appointment, add it to a calendar app, replace it in the calendar app, and remove it from the calendar app. We'll also show how to display a time span for a calendar app and create an appointment-recurrence object.

Create an appointment and apply data to it

Create a [Windows.ApplicationModel.Appointments.Appointment](#) object and assign it to a variable. Then, apply to the **Appointment** the appointment properties that were supplied through the UI by a user.

C#

```
private void Create_Click(object sender, RoutedEventArgs e)
{
    bool isAppointmentValid = true;
    var appointment = new
Windows.ApplicationModel.Appointments.Appointment();

    // StartTime
    var date = StartTimeDatePicker.Date;
    var time = StartTimeTimePicker.Time;
    var timeZoneOffset = TimeZoneInfo.Local.GetUtcOffset(DateTime.Now);
    var startTime = new DateTimeOffset(date.Year, date.Month, date.Day,
time.Hours, time.Minutes, 0, timeZoneOffset);
    appointment.StartTime = startTime;

    // Subject
    appointment.Subject = SubjectTextBox.Text;

    if (appointment.Subject.Length > 255)
    {
        isAppointmentValid = false;
        ResultTextBlock.Text = "The subject cannot be greater than 255
characters.";
    }

    // Location
    appointment.Location = LocationTextBox.Text;

    if (appointment.Location.Length > 32768)
    {
        isAppointmentValid = false;
        ResultTextBlock.Text = "The location cannot be greater than 32,768
```

```

characters.";
    }

    // Details
    appointment.Details = DetailsTextBox.Text;

    if (appointment.Details.Length > 1073741823)
    {
        isAppointmentValid = false;
        ResultTextBlock.Text = "The details cannot be greater than
1,073,741,823 characters.";
    }

    // Duration
    if (DurationComboBox.SelectedIndex == 0)
    {
        // 30 minute duration is selected
        appointment.Duration = TimeSpan.FromMinutes(30);
    }
    else
    {
        // 1 hour duration is selected
        appointment.Duration = TimeSpan.FromHours(1);
    }

    // All Day
    appointment.AllDay = AllDayCheckBox.IsChecked.Value;

    // Reminder
    if (ReminderCheckBox.IsChecked.Value)
    {
        switch (ReminderComboBox.SelectedIndex)
        {
            case 0:
                appointment.Reminder = TimeSpan.FromMinutes(15);
                break;
            case 1:
                appointment.Reminder = TimeSpan.FromHours(1);
                break;
            case 2:
                appointment.Reminder = TimeSpan.FromDays(1);
                break;
        }
    }

    //Busy Status
    switch (BusyStatusComboBox.SelectedIndex)
    {
        case 0:
            appointment.BusyStatus =
Windows.ApplicationModel.Appointments.AppointmentBusyStatus.Busy;
            break;
        case 1:
            appointment.BusyStatus =
Windows.ApplicationModel.Appointments.AppointmentBusyStatus.Tentative;

```

```

        break;
    case 2:
        appointment.BusyStatus =
Windows.ApplicationModel.Appointments.AppointmentBusyStatus.Free;
        break;
    case 3:
        appointment.BusyStatus =
Windows.ApplicationModel.Appointments.AppointmentBusyStatus.OutOfOffice;
        break;
    case 4:
        appointment.BusyStatus =
Windows.ApplicationModel.Appointments.AppointmentBusyStatus.WorkingElsewhere
;
        break;
    }

    // Sensitivity
    switch (SensitivityComboBox.SelectedIndex)
    {
        case 0:
            appointment.Sensitivity =
Windows.ApplicationModel.Appointments.AppointmentSensitivity.Public;
            break;
        case 1:
            appointment.Sensitivity =
Windows.ApplicationModel.Appointments.AppointmentSensitivity.Private;
            break;
    }

    // Uri
    if (UriTextBox.Text.Length > 0)
    {
        try
        {
            appointment.Uri = new System.Uri(UriTextBox.Text);
        }
        catch (Exception)
        {
            isValidAppointment = false;
            ResultTextBlock.Text = "The Uri provided is invalid.";
        }
    }

    // Organizer
    // Note: Organizer can only be set if there are no invitees added to
    this appointment.
    if (OrganizerRadioButton.IsChecked.Value)
    {
        var organizer = new
Windows.ApplicationModel.Appointments.AppointmentOrganizer();

        // Organizer Display Name
        organizer.DisplayName = OrganizerDisplayNameTextBox.Text;

        if (organizer.DisplayName.Length > 256)

```



```

    {
        isAppointmentValid = false;
        ResultTextBlock.Text = "The organizer display name cannot be
greater than 256 characters.";
    }
    else
    {
        // Organizer Address (for example, Email Address)
        organizer.Address = OrganizerAddressTextBox.Text;

        if (organizer.Address.Length > 321)
        {
            isAppointmentValid = false;
            ResultTextBlock.Text = "The organizer address cannot be
greater than 321 characters.";
        }
        else if (organizer.Address.Length == 0)
        {
            isAppointmentValid = false;
            ResultTextBlock.Text = "The organizer address must be
greater than 0 characters.";
        }
        else
        {
            appointment.Organizer = organizer;
        }
    }
}

// Invitees
// Note: If the size of the Invitees list is not zero, then an Organizer
cannot be set.
if (InviteeRadioButton.IsChecked.Value)
{
    var invitee = new
Windows.ApplicationModel.Appointments.AppointmentInvitee();

    // Invitee Display Name
    invitee.DisplayName = InviteeDisplayNameTextBox.Text;

    if (invitee.DisplayName.Length > 256)
    {
        isAppointmentValid = false;
        ResultTextBlock.Text = "The invitee display name cannot be
greater than 256 characters.";
    }
    else
    {
        // Invitee Address (for example, Email Address)
        invitee.Address = InviteeAddressTextBox.Text;

        if (invitee.Address.Length > 321)
        {
            isAppointmentValid = false;
            ResultTextBlock.Text = "The invitee address cannot be

```

```

greater than 321 characters.";
    }
    else if (invitee.Address.Length == 0)
    {
        isAppointmentValid = false;
        ResultTextBlock.Text = "The invitee address must be greater
than 0 characters.";
    }
    else
    {
        // Invitee Role
        switch (RoleComboBox.SelectedIndex)
        {
            case 0:
                invitee.Role =
Windows.ApplicationModel.Appointments.AppointmentParticipantRole.RequiredAtt
endee;
                break;
            case 1:
                invitee.Role =
Windows.ApplicationModel.Appointments.AppointmentParticipantRole.OptionalAtt
endee;
                break;
            case 2:
                invitee.Role =
Windows.ApplicationModel.Appointments.AppointmentParticipantRole.Resource;
                break;
        }

        // Invitee Response
        switch (ResponseComboBox.SelectedIndex)
        {
            case 0:
                invitee.Response =
Windows.ApplicationModel.Appointments.AppointmentParticipantResponse.None;
                break;
            case 1:
                invitee.Response =
Windows.ApplicationModel.Appointments.AppointmentParticipantResponse.Tentati
ve;
                break;
            case 2:
                invitee.Response =
Windows.ApplicationModel.Appointments.AppointmentParticipantResponse.Accepte
d;
                break;
            case 3:
                invitee.Response =
Windows.ApplicationModel.Appointments.AppointmentParticipantResponse.Decline
d;
                break;
            case 4:
                invitee.Response =
Windows.ApplicationModel.Appointments.AppointmentParticipantResponse.Unknown
;

```

```

        break;
    }

    appointment.Invitees.Add(invitee);
}
}

if (isAppointmentValid)
{
    ResultTextBlock.Text = "The appointment was created successfully and is valid.";
}
}

```

Add an appointment to the user's calendar

Create a [Windows.ApplicationModel.Appointments.Appointment](#) object and assign it to a variable. Then, call the [AppointmentManager.ShowAddAppointmentAsync\(Appointment, Rect, Placement\)](#) method to show the default appointments provider add-appointment UI, to enable the user to add an appointment. If the user clicked **Add**, the sample prints the appointment identifier that **ShowAddAppointmentAsync** returned.

C#

```

private async void Add_Click(object sender, RoutedEventArgs e)
{
    // Create an Appointment that should be added the user's appointments provider app.
    var appointment = new
Windows.ApplicationModel.Appointments.Appointment();

    // Get the selection rect of the button pressed to add this appointment
    var rect = GetElementRect(sender as FrameworkElement);

    // ShowAddAppointmentAsync returns an appointment id if the appointment given was added to the user's calendar.
    // This value should be stored in app data and roamed so that the appointment can be replaced or removed in the future.
    // An empty string return value indicates that the user canceled the operation before the appointment was added.
    String appointmentId = await
Windows.ApplicationModel.Appointments.AppointmentManager.ShowAddAppointmentAsync(
        appointment, rect,
Windows.UI.Popups.Placement.Default);
    if (appointmentId != String.Empty)
    {
        ResultTextBlock.Text = "Appointment Id: " + appointmentId;
    }
}

```

```

    }
    else
    {
        ResultTextBlock.Text = "Appointment not added.";
    }
}

```

Note For Windows Phone Store apps, [ShowAddAppointment](#) functions just like [ShowEditNewAppointment](#) in that the dialog displayed for adding the appointment is editable.

Replace an appointment in the user's calendar

Create a [Windows.ApplicationModel.Appointments.Appointment](#) object and assign it to a variable. Then, call the appropriate [AppointmentManager.ShowReplaceAppointmentAsync](#) method to show the default appointments provider replace-appointment UI to enable the user to replace an appointment. The user also provides the appointment identifier that they want to replace. This identifier was returned from [AppointmentManager.ShowAddAppointmentAsync](#). If the user clicked **Replace**, the sample prints that it updated that appointment identifier.

C#

```

private async void Replace_Click(object sender, RoutedEventArgs e)
{
    // The appointment id argument for ReplaceAppointmentAsync is typically
    // retrieved from AddAppointmentAsync and stored in app data.
    String appointmentIdOfAppointmentToReplace = AppointmentIdTextBox.Text;

    if (String.IsNullOrEmpty(appointmentIdOfAppointmentToReplace))
    {
        ResultTextBlock.Text = "The appointment id cannot be empty";
    }
    else
    {
        // The Appointment argument for ReplaceAppointmentAsync should
        // contain all of the Appointment's properties including those that may have
        // changed.
        var appointment = new
Windows.ApplicationModel.Appointments.Appointment();

        // Get the selection rect of the button pressed to replace this
        // appointment
        var rect = GetElementRect(sender as FrameworkElement);

        // ReplaceAppointmentAsync returns an updated appointment id when
        // the appointment was successfully replaced.
    }
}

```

```

        // The updated id may or may not be the same as the original one
        retrieved from AddAppointmentAsync.
        // An optional instance start time can be provided to indicate that
        a specific instance on that date should be replaced
        // in the case of a recurring appointment.
        // If the appointment id returned is an empty string, that indicates
        that the appointment was not replaced.
        String updatedAppointmentId;
        if (InstanceStartDateCheckBox.IsChecked.Value)
        {
            // Replace a specific instance starting on the date provided.
            var instanceStartDate = InstanceStartDateDatePicker.Date;
            updatedAppointmentId = await
Windows.ApplicationModel.Appointments.AppointmentManager.ShowReplaceAppointmentAsync(
                                appointmentIdOfAppointmentToReplace,
appointment, rect, Windows.UI.Popups.Placement.Default, instanceStartDate);
        }
        else
        {
            // Replace an appointment that occurs only once or in the case
            of a recurring appointment, replace the entire series.
            updatedAppointmentId = await
Windows.ApplicationModel.Appointments.AppointmentManager.ShowReplaceAppointmentAsync(
                                appointmentIdOfAppointmentToReplace,
appointment, rect, Windows.UI.Popups.Placement.Default);
        }

        if (updatedAppointmentId != String.Empty)
        {
            ResultTextBlock.Text = "Updated Appointment Id: " +
updatedAppointmentId;
        }
        else
        {
            ResultTextBlock.Text = "Appointment not replaced.";
        }
    }
}

```

Remove an appointment from the user's calendar

Call the appropriate [AppointmentManager.ShowRemoveAppointmentAsync](#) method to show the default appointments provider remove-appointment UI, to enable the user to remove an appointment. The user also provides the appointment identifier that they want to remove. This identifier was returned from

[AppointmentManager.ShowAddAppointmentAsync](#). If the user clicked **Delete**, the sample prints that it removed the appointment specified by that appointment identifier.

C#

```
private async void Remove_Click(object sender, RoutedEventArgs e)
{
    // The appointment id argument for ShowRemoveAppointmentAsync is
    // typically retrieved from AddAppointmentAsync and stored in app data.
    String appointmentId = AppointmentIdTextBox.Text;

    // The appointment id cannot be null or empty.
    if (String.IsNullOrEmpty(appointmentId))
    {
        ResultTextBlock.Text = "The appointment id cannot be empty";
    }
    else
    {
        // Get the selection rect of the button pressed to remove this
        // appointment
        var rect = GetElementRect(sender as FrameworkElement);

        // ShowRemoveAppointmentAsync returns a boolean indicating whether
        // or not the appointment related to the appointment id given was removed.
        // An optional instance start time can be provided to indicate that
        // a specific instance on that date should be removed
        // in the case of a recurring appointment.
        bool removed;
        if (InstanceStartDateCheckBox.IsChecked.Value)
        {
            // Remove a specific instance starting on the date provided.
            var instanceStartDate = InstanceStartDateDatePicker.Date;
            removed = await
Windows.ApplicationModel.Appointments.AppointmentManager.ShowRemoveAppointmentAsync(
                appointmentId, rect,
Windows.UI.Popups.Placement.Default, instanceStartDate);
        }
        else
        {
            // Remove an appointment that occurs only once or in the case of
            // a recurring appointment, replace the entire series.
            removed = await
Windows.ApplicationModel.Appointments.AppointmentManager.ShowRemoveAppointmentAsync(
                appointmentId, rect,
Windows.UI.Popups.Placement.Default);
        }

        if (removed)
        {
            ResultTextBlock.Text = "Appointment removed";
        }
        else
    }
}
```

```

    {
        ResultTextBlock.Text = "Appointment not removed";
    }
}

```

Show a time span for the appointments provider

Call the [AppointmentManager.ShowTimeFrameAsync](#) method to show a specific time span for the default appointments provider's primary UI if the user clicked **Show**. The sample prints that the default appointments provider appeared on screen.

C#

```

private async void Show_Click(object sender, RoutedEventArgs e)
{
    var dateToShow = new DateTimeOffset(2015, 6, 12, 18, 32, 0, 0,
    TimeSpan.FromHours(-8));
    var duration = TimeSpan.FromHours(1);
    await
    Windows.ApplicationModel.Appointments.AppointmentManager.ShowTimeFrameAsync(
    dateToShow, duration);
    ResultTextBlock.Text = "The default appointments provider should have
    appeared on screen.";
}

```

Create an appointment-recurrence object and apply data to it

Create an [Windows.ApplicationModel.Appointments.AppointmentRecurrence](#) object and assign it to a variable. Then, apply to the **AppointmentRecurrence** the recurrence properties that were supplied through the UI by a user.

C#

```

private void Create_Click(object sender, RoutedEventArgs e)
{
    bool isRecurrenceValid = true;
    var recurrence = new
    Windows.ApplicationModel.Appointments.AppointmentRecurrence();

    // Unit
    switch (UnitComboBox.SelectedIndex)
    {

```

```

        case 0:
            recurrence.Unit =
Windows.ApplicationModel.Appointments.AppointmentRecurrenceUnit.Daily;
            break;
        case 1:
            recurrence.Unit =
Windows.ApplicationModel.Appointments.AppointmentRecurrenceUnit.Weekly;
            break;
        case 2:
            recurrence.Unit =
Windows.ApplicationModel.Appointments.AppointmentRecurrenceUnit.Monthly;
            break;
        case 3:
            recurrence.Unit =
Windows.ApplicationModel.Appointments.AppointmentRecurrenceUnit.MonthlyOnDay
;
            break;
        case 4:
            recurrence.Unit =
Windows.ApplicationModel.Appointments.AppointmentRecurrenceUnit.Yearly;
            break;
        case 5:
            recurrence.Unit =
Windows.ApplicationModel.Appointments.AppointmentRecurrenceUnit.YearlyOnDay;
            break;
    }

    // Occurrences
    // Note: Occurrences and Until properties are mutually exclusive.
    if (OccurrencesRadioButton.IsChecked.Value)
    {
        recurrence.Occurrences = (uint)OccurrencesSlider.Value;
    }

    // Until
    // Note: Until and Occurrences properties are mutually exclusive.
    if (UntilRadioButton.IsChecked.Value)
    {
        recurrence.Until = UntilDatePicker.Date;
    }

    // Interval
    recurrence.Interval = (uint)IntervalSlider.Value;

    // Week of the month
    switch (WeekOfMonthComboBox.SelectedIndex)
    {
        case 0:
            recurrence.WeekOfMonth =
Windows.ApplicationModel.Appointments.AppointmentWeekOfMonth.First;
            break;
        case 1:
            recurrence.WeekOfMonth =
Windows.ApplicationModel.Appointments.AppointmentWeekOfMonth.Second;
            break;
    }

```



```

        case 2:
            recurrence.WeekOfMonth =
Windows.ApplicationModel.Appointments.AppointmentWeekOfMonth.Third;
            break;
        case 3:
            recurrence.WeekOfMonth =
Windows.ApplicationModel.Appointments.AppointmentWeekOfMonth.Fourth;
            break;
        case 4:
            recurrence.WeekOfMonth =
Windows.ApplicationModel.Appointments.AppointmentWeekOfMonth.Last;
            break;
    }

    // Days of the Week
    // Note: For Weekly, MonthlyOnDay or YearlyOnDay recurrence unit values,
    at least one day must be specified.
    if (SundayCheckBox.IsChecked.Value) { recurrence.DaysOfWeek |=
Windows.ApplicationModel.Appointments.AppointmentDaysOfWeek.Sunday; }
    if (MondayCheckBox.IsChecked.Value) { recurrence.DaysOfWeek |=
Windows.ApplicationModel.Appointments.AppointmentDaysOfWeek.Monday; }
    if (TuesdayCheckBox.IsChecked.Value) { recurrence.DaysOfWeek |=
Windows.ApplicationModel.Appointments.AppointmentDaysOfWeek.Tuesday; }
    if (WednesdayCheckBox.IsChecked.Value) { recurrence.DaysOfWeek |=
Windows.ApplicationModel.Appointments.AppointmentDaysOfWeek.Wednesday; }
    if (ThursdayCheckBox.IsChecked.Value) { recurrence.DaysOfWeek |=
Windows.ApplicationModel.Appointments.AppointmentDaysOfWeek.Thursday; }
    if (FridayCheckBox.IsChecked.Value) { recurrence.DaysOfWeek |=
Windows.ApplicationModel.Appointments.AppointmentDaysOfWeek.Friday; }
    if (SaturdayCheckBox.IsChecked.Value) { recurrence.DaysOfWeek |=
Windows.ApplicationModel.Appointments.AppointmentDaysOfWeek.Saturday; }

    if (((recurrence.Unit ==
Windows.ApplicationModel.Appointments.AppointmentRecurrenceUnit.Weekly) ||
        (recurrence.Unit ==
Windows.ApplicationModel.Appointments.AppointmentRecurrenceUnit.MonthlyOnDay
) ||
        (recurrence.Unit ==
Windows.ApplicationModel.Appointments.AppointmentRecurrenceUnit.YearlyOnDay)
) &&
        (recurrence.DaysOfWeek ==
Windows.ApplicationModel.Appointments.AppointmentDaysOfWeek.None))
    {
        isRecurrenceValid = false;
        ResultTextBlock.Text = "The recurrence specified is invalid. For
Weekly, MonthlyOnDay or YearlyOnDay recurrence unit values, " +
                                "at least one day must be specified.";
    }

    // Month of the year
    recurrence.Month = (uint)MonthSlider.Value;

    // Day of the month
    recurrence.Day = (uint)DaySlider.Value;

```

```

    if (isRecurrenceValid)
    {
        ResultTextBlock.Text = "The recurrence specified was created successfully and is valid.";
    }
}

```

Add a new editable appointment

[ShowEditNewAppointmentAsync](#) works just like [ShowAddAppointmentAsync](#) except that the dialog for adding the appointment is editable so that the user can modify the appointment data before saving it.

C#

```

private async void AddAndEdit_Click(object sender, RoutedEventArgs e)
{
    // Create an Appointment that should be added the user' s appointments provider app.
    var appointment = new
Windows.ApplicationModel.Appointments.Appointment();

    appointment.StartTime = DateTime.Now + TimeSpan.FromDays(1);
    appointment.Duration = TimeSpan.FromHours(1);
    appointment.Location = "Meeting location";
    appointment.Subject = "Meeting subject";
    appointment.Details = "Meeting description";
    appointment.Reminder = TimeSpan.FromMinutes(15); // Remind me 15 minutes prior

    // ShowAddAppointmentAsync returns an appointment id if the appointment given was added to the user' s calendar.
    // This value should be stored in app data and roamed so that the appointment can be replaced or removed in the future.
    // An empty string return value indicates that the user canceled the operation before the appointment was added.
    String appointmentId =
        await
Windows.ApplicationModel.Appointments.AppointmentManager.ShowEditNewAppointmentAsync(appointment);

    if (appointmentId != String.Empty)
    {
        ResultTextBlock.Text = "Appointment Id: " + appointmentId;
    }
    else
    {
        ResultTextBlock.Text = "Appointment not added.";
    }
}

```

Show appointment details

[ShowAppointmentDetailsAsync](#) causes the system to show details for the specified appointment. An app that implements app calendars may choose to be activated to show details for appointments in calendars it owns. Otherwise, the system will show the appointment details. An overload of the method that accepts a start date argument is provided to show details for an instance of a recurring appointment.

C#

```
private async void ShowAppointmentDetails-Click(object sender,
RoutedEventArgs e)
{
    if (instanceStartTime == null)
    {
        await
Windows.ApplicationModel.Appointments.AppointmentManager.ShowAppointmentDetailsAsync(
        currentAppointment.LocalId);
    }
    else
    {
        // Specify a start time to show an instance of a recurring
        appointment
        await
Windows.ApplicationModel.Appointments.AppointmentManager.ShowAppointmentDetailsAsync(
        currentAppointment.LocalId, instanceStartTime);
    }
}
```

Summary and next steps

Now you have a basic understanding of how to manage appointments. Download the [Universal Windows app samples](#) from GitHub to see more examples of how to manage appointments.

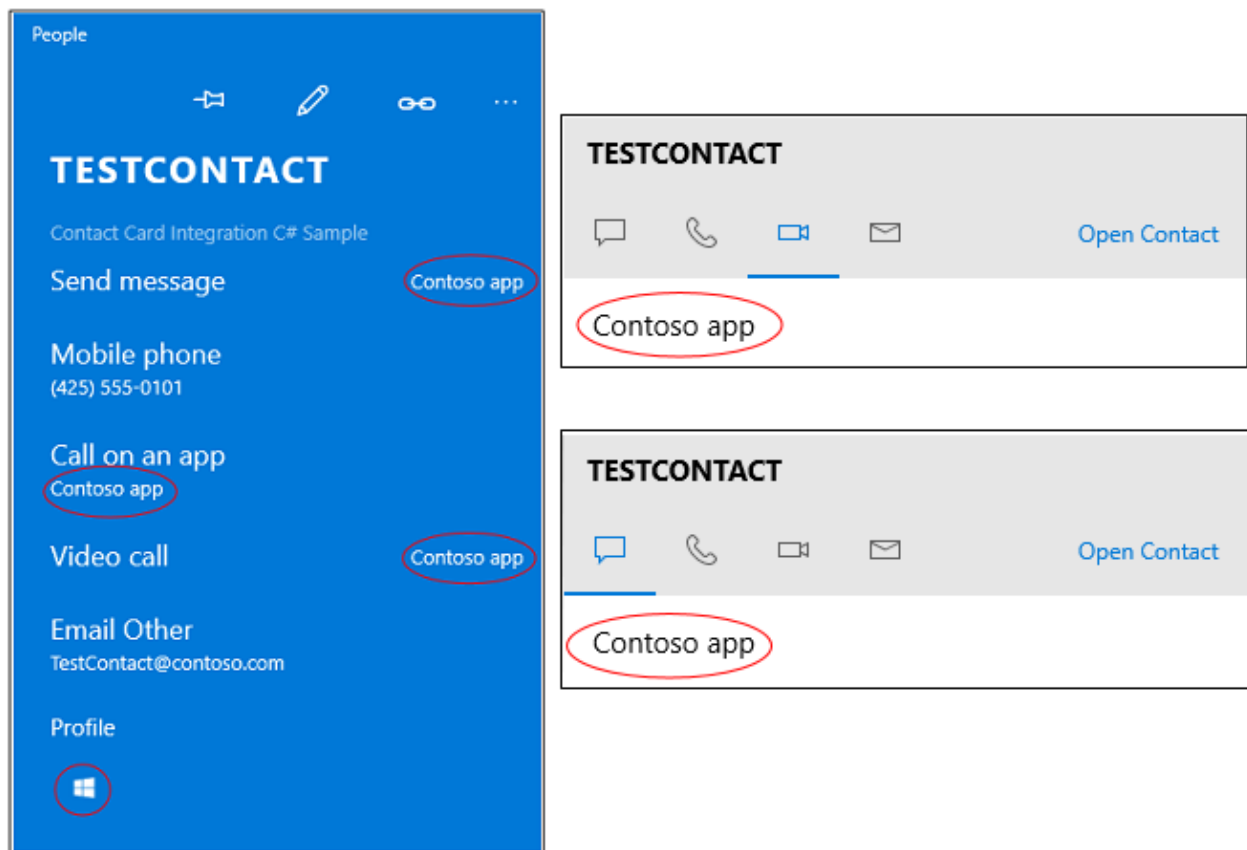
Related topics

- [Appointments API sample](#)

Connect your app to actions on a contact card

Article • 10/20/2022

Your app can appear next to actions on a contact card or mini contact card. Users can choose your app to perform an action such as open a profile page, place a call, or send a message.



To get started, find existing contacts or create new ones. Next, create an *annotation* and a few package manifest entries to describe which actions your app supports. Then, write code that perform the actions.

For a more complete sample, see [Contact Card Integration Sample](#).

Find or create a contact

If your app helps people connect with others, search Windows for contacts and then annotate them. If your app manages contacts, you can add them to a Windows contact list and then annotate them.

Find a contact

Find contacts by using a name, email address, or phone number.

C#

```
ContactStore contactStore = await ContactManager.RequestStoreAsync();

IReadOnlyList<Contact> contacts = null;

contacts = await contactStore.FindContactsAsync(emailAddress);

Contact contact = contacts[0];
```

Create a contact

If your app is more like an address book, create contacts and then add them to a contact list.

C#

```
Contact contact = new Contact();
contact.FirstName = "TestContact";

ContactEmail email = new ContactEmail();
email.Address = "TestContact@contoso.com";
email.Kind = ContactEmailKind.Other;
contact.Emails.Add(email);

ContactPhone phone = new ContactPhone();
phone.Number = "4255550101";
phone.Kind = ContactPhoneKind.Mobile;
contact.Phones.Add(phone);

ContactStore store = await

ContactManager.RequestStoreAsync(ContactStoreAccessType.AppContactsReadWrite
);

ContactList contactList;

IReadOnlyList<ContactList> contactLists = await
store.FindContactListsAsync();

if (0 == contactLists.Count)
    contactList = await store.CreateContactListAsync("TestContactList");
else
    contactList = contactLists[0];

await contactList.SaveContactAsync(contact);
```